דוח משוב לסטודנט

מזהה סטודנט	[redacted]
מספר מבחן	0001
תאריך בחינה	יום רביע', 5 בפבר' 2025
מזהה קורס	20594
שם קורס	מערכות הפעלה
מרצה	אריה לויטן

תשובות נכונות (שאלות סגורות)	ניקוד שאלות סגורות	ניקוד שאלות פתוחות	ציון מבחן מקורי	**ציון מבחן סופי**
5	25.00	67.00	92.00	**92.00**

סיכום

מספר שאלה בטופס זה	מספר השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור	ניקוד	ניקוד מירבי	התשובה שסומנה	התשובה הנכונה
1	1	3	5.00	5.00	3 ✓	3
2	2	2	5.00	5.00	2 ✓	2
3	3	4	0	5.00	4 ×	1, 3
4	4	4	5.00	5.00	4 ✓	4
5	5	3	5.00	5.00	3 ✓	3
6	6	1	5.00	5.00	1 ✓	1
7	7		25.00	25.00		
8	8		15.00	15.00		
9	9		12.00	15.00		
10	10		5.00	5.00		
11	11		5.00	5.00		
12	12		5.00	5.00		
13	13			5.00		

✓ תשובה נכונה ✓ תשובה חלקית × תשובה שגויה ⊘ לא נענתה

POWERED BY
Tomax - Think Academy

האוניברסיטה
הפתוחה

[redacted]

דף תשובות

מספר גרסה	מספר תעודת זהות	מספר הקורס	תאריך בחינה
0001	[redacted]	20594	050225

0	●	●	●	○
1	○	○	○	●
2	○	○	○	○
3	○	○	○	○
4	○	○	○	○
5	○	○	○	○
6	○	○	○	○
7	○	○	○	○
8	○	○	○	○
9	○	○	○	○

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימון, יש לסמן X על התשובה הלא נכונה - ☒
ולהשאיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה,
ולהשחיר מחדש את סימן ה- ☒ באופן שיכסה המחיקה - ●

שאלה / תשובה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
א	○	○	☒	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	●	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	●	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

שאלה / תשובה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

ש מ [redacted]

מספר סידורי: 8 [redacted]

[redacted]

מספר תעודת זהות

יש לרשום את כל תשע הספרות

האוניברסיטה הפתוחה

ז' בשבט תשפ"ה

5 בפברואר 2025

סמסטר 2025א

20594 / 4

מס' שאלון - 512

מס' מועד 81

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם
בתשובה יחידה הנכונה ביותר מבין התשובות.
את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.

ג. בחלקים ב' ו ג' מופיעות שאלות פתוחות.
ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ.
אין חובה להשתמש בכל השורות המוקצות לצורך התשובות,
אך אין לחרוג מהמקום המוקצה.

חומר עזר:

כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א׳ יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב׳ ו ג׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא** ו**בקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

מהי שיטת התזמון SCHEDULING המתאימה ביותר בשביל להשיג את תגובתיות RESPONSIVENESS (תגובה ראשונית מהירה) באפליקציות אינטראקטיביות?

א. shortest job first

ב. תזמון ע״פ עדיפויות קבועות

[✓ ג.] Round robin עם quantum קצר

ד. Round robin עם quantum ארוך

שאלה 2

מה נכון לגבי HYPERVISOR מסוג 2 בתצורת HOSTED ?

א. הקצאת כל משאבי החומרה hardware resources נשלטת **ע״י מערכת ההפעלה האורחת**.

[✓ ב.] מערכת הפעלה אורחת GUEST רצה במצב משתמש.

ג. בתוך המכונה הוירטואלית ניתן להתקין רק את מערכת הפעלה האורחת GUEST שתואמת את המערכת המארחת HOST .

ד. לשתי מערכות- אורחת ומארחת חייב להיות אותו גרעין.

שאלה 3

מתברר כי במערכת מסוימת ישנה בעיה **חוזרת** של deadlock (קיפאון, מבוי סתום) בזמן הקצאת המשאבים.

באיזו מדיניות ניתן לנקוט כדי לפתור את הבעיה?

א. מדיניות בה כל **סוג** של משאב יקבל מספר סידורי, וכל תהליך צריך לבקש את המשאבים שהוא רוצה, עפ"י סדר עולה של המספרים הסידוריים. הכוונה שמי שיש את 7 לא יכול לבקש 2?

ב. מדיניות שבה לאחר כל פרק זמן קבוע, מופעלת בדיקה אם יש מבוי סתום, ובמידה ומתברר שישנו מבוי סתום – תבוטלנה כל ההקצאות של כל המשאבים עבור כל התהליכים. ← שובר את תנאי החזקה והמתנה.

ג. מדיניות הקובעת שהקצאת משאב תבוצע אך ורק בתוך קטע קריטי, הכולל בתוכו גם תהליך בדיקה שלא ייווצר מבוי סתום כתוצאה מההקצאה. ← ~~כי לא מונע, עדיין יש קיפאון למרות הכל בקטע קריטי.~~

~~מונע תנאי~~ אומר שיהיה מצב קטוע! מה טוב!

(גורם - מונע הדדיות של החזקת משאבים. מקיים תנאי קיפאון, לכן לא טוב!) כמו בנקאי.

[✓ ד.] כל התשובות נכונות.

שאלה 4

טבלת הדפים של תהליך במערכת עם זיכרון וירטואלי נראית כך. כל המספרים הם דצימליים, מתחילים מאפס, וכל הכתובות הן כתובות של ביית בזיכרון. גודל הדף הוא 4096 בתים(4Kbyte).

2^{12}

$2^2 \cdot 2^{10}$

Frame Number	Valid bit	Page Number
1	1	0
2	1	1
0	0	2
0	1	3
3	0	4
0	0	5

לאיזו כתובת פיזית, אם ניתן לחשב, תמופה הכתובת הוירטואלית הבאה: 9000 ?

א. 9000

ב. 808

ג. 4096

[✓ ד.] לא ניתן לחשב

valid = 0

offset 12

$\boxed{0010}\,\underbrace{0011}\,\underbrace{0010}\,\underbrace{1000}$

$\downarrow$

2

off = 808

נא לא לכתוב בשוליים

שאלה 5

ש. רק ליבה אחת
הליכות האחרות ברגע).

ידוע לנו כי תהליכון מסויים מבצע קוד שיש להגדירו כקטע קריטי. A
כדי למנוע בעיה של סנכרון, הוצע למנוע מן המעבד לבצע פסיקות במהלך ביצוע תהליכון זה – כך, שמרגע שהתהליכון יקבל את המעבד הוא יתבצע ללא הפסקה באמצע.
האם ההצעה אכן מונעת בעיה של סינכרון?

א. ההצעה אכן מונעת בעיה של סינכרון.

ב. ההצעה מבטיחה מניעה הדדית והמתנה מוגבלת אבל לא מבטיחה התקדמות - ולכן היא לא פותרת את הבעיה במלואה.

[✓ ג.] אם מדובר במערכת עם ליבה אחת, אכן ההצעה תימנע את הבעיה. אם מדובר במערכת מרובת ליבות – הבעיה עדיין עלולה להתרחש. ט תהליכים יוכלו לחלק בקטע הקריטי על מעבד אחר על ליבה אחרת.

ד. אם במהלך ביצוע התהליכון תתבצע הקצאת משאבים, הבעיה עדיין עלולה להתרחש. במידה ולא תתבצע הקצאה, הבעיה תימנע.

שאלה 6

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים :

- גודל הבלוק במערכת הקבצים הוא 2 Kbytes
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

2^{11}
2^{2}

כמה בלוק מצביעים?

$\frac{2^{11}}{2^{2}} = 2^{9}$ מצביעים
512

חשבו מה הגודל המרבי של קובץ שניתן לאכסן ב $\boxed{2023}$ בלוקים בסה"כ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו) .

הערה : לצורך החישוב 1 KByte = 1024 Byte , 1 MByte = 1024 KByte

[✓ א.] 4036 Kbytes

ב. 4040 Kbytes

ג. 4000Kbytes

ד. 4032Kbytes

בלוקי מידע	מס בלוקים	אינדקס	
10	10		
10	11	✱	indirect
522	523		
522	524	✱	double
1034	1037		
1546	1550		

המשך הבחינה בעמוד הבא

2018 X2 = 4036

ind ✱
double ✱
doble שי
double שי
double שי

כן חצה לכתוב את הערך 1
אם הערך הקודם היה 0.

~~אם מת~~
קטן while:
אם הערך 0-
ננסה לכתוב. אם הצלחנו-
נעשה break!

```
while(true)
  if (LL(m)==0)
    if(SC(1,m)==1):
      break;
```

נחכה עד שהמנעול יתפנה, נהפוך אותו ל-1
ונוכל להכנס.

v=LL(p) שומר את מה שבתוך p ל-v.

r=SC(v,p) מנסה לכתוב את v לתוך p.

אם מצליח → 1, אם נכשל → 0.

אם הפעולה האחרונה היתה LL(p)

חלק ב (55 נקודות)

ענו על השאלות 7-9.

שאלה 7 (25 נקודות)

א. (10 נק׳)

חלק ממעבדים (לדוגמא ARM) תומכים בשתי פקודות מכונה אטומיות. לתיאום בין תהליכונים (THREADS).

פקודת מכונה אחת LL(p) מחזירה את הערך השמור בכתובת p (שקול לאופרטור * על מצביע). כלומר v=LL(p) שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v.

הפקודה השניה SC תשמור את הערך בכתובת זיכרון רק אם מאז ביצוע פקודת LL על אותה הכתובת לא היו בה שמירות ע״י תהליכוניים אחרים. במקרה של הצלחה סטטוס סיום הפקודה שווה ל 1 ובמקרה של כישלון ל 0. ההוראה $r = SC(v,p)$ מנסה לכתוב את הערך v לכתובת p בזיכרון. דומה ל- TSL

אם הפעולה האחרונה של התהליכון הנוכחי על הכתובת p הייתה LL, ואף תהליכון אחר לא כתב ערך לכתובת p מאז שהתהליכון הנוכחי ביצע LL(p) , הכתיבה מצליחה והפקודה מחזירה 1.

בכל מקרה אחר, כלומר אם תהליכון לא ביצע LL , או אם תהליכון אחר כתב לכתובת p מאז שהוא ביצע (בין אם בעזרת LL ובין אם בעזרת השמה בכל דרך), הכתיבה נכשלת (הערך v לא נכתב לכתובת p) והערך המוחזר הוא 0 . במילים אחרות, הכתיבה מצליחה רק אם הערך שכתוב בזיכרון הוא הערך שהתהליכון בעצמו קרא בעזרת ה LL ואף תהליכון אחר לא כתב לתוכו מאז.

ממש שגרות lock(m) ו unlock(m) למניעה הדדית, כאשר m הוא מצביע למספר שלם שמייצג את המנעול MUTEX.

המימוש צריך להתבסס על 2 מקרואים(MACROS) התואמים את הפקודות בשאלה:

- v=LL(p) (שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v)
- $r = SC(v,p)$ (המנסה לכתוב את הערך v לכתובת p בזיכרון ומחזיר את סטטוס סיום) → While

המימוש חייב להבטיח את **מניעה הדדית והתקדמות**, אך **אינו צריך להבטיח המתנה מוגבלת** (העדר הרעבה). אפשר break, continue

תשובה:

```
lock(m):
        while(true):
            if(LL(m)==0):
                if(SC(1,m)==1):
                    break;

unlock(m):
        while(SC(0,m)!=1):
            v=LL(m)
```

דומה ל- TSL קצת.

אם המנעול פנוי →

אנחנו תפסנו ראשונים →

נשמור ונתחיל את הקטע הקריטי!

מיותר, אפשר בהשמה רגילה!

אפשר נפתח את המנעול ע״י השמת 0

עד שנצליח לכתוב לתוך את ה-0,

נמשיך בלולאה לנסות (כי אולי תהליכים אחרים שמנסים להיכנס עשו LL(m) לפניי)

נא לא לכתוב בשוליים

ב. (15 נקי)

על בסיס שימוש בפונקציות נעילה ושחרור מסעיף א', כתבו פסודו-קוד המדמה פתרון לבעיה הבאה:

קיים קטע כביש צר חד נתיבי שהתנועה בו מותרת בכל רגע נתון רק בכיוון אחד. כל עוד שיש תנועה באותו הכיוון, מכוניות נוספות יכולות לנסוע באותו הכיוון. אבל מכוניות שרוצות לנסוע מהצד השני של הקטע הצר בכיוון הפוך, צריכות להמתין עד שהתנועה הקיימת תיפסק. כל מכונית ממומשת כ תהליכון THREAD . מותר להשתמש אך ורק במשתנים הבאים: 3 מנעולים m1, m2, m3 ושני מונים c1, c2 .

צריך לממש רק את פרוטוקול הסנכרון ואין צורך לממש נסיעה עצמה, מספיק לכתוב את זה בהערה.

תשובה והסבר:

m3 - ~~[illegible]~~ מניעת התנגשות בין כיוונים

m2 - מגן על c2 וכן מבטיח עדיפות לכיוון הראשי.

m1 - מגן על c1.

אבל איפשר הרבה כותבים! צריך לממש שהתנועה תיפסק רק אחרי כי רק "כותב" אחד בכיוון הנגדי.

ניתן להקביל את הבעיה הנתונה לבעיית קוראים-כותבים, כאשר נוסע מהצד השני הוא כותב, ויכול להתבצע רק אם אין תנועה בכיוון הראשי, ואילו תנועה בכיוון הראשי מותרת כשיש כבר תנועה- כמו קוראים בו זמנית.

~~other_directions~~ $c_1, c_2, m_1, m_2, m_3 = 0$

```
drive-1: // הכיוון הראשי
lock(m1);
c1++;
if (c1==1):
    lock(m3)
unlock(m1)
// drive
lock(m1);
c1--;
if(c1==0)
    unlock(m3)
unlock(m1)
```

```
drive-2: / הכיוון הנגדי
lock(m2)
c2++
if(c2==1)
    lock(m3)
unlock(m2)
// drive
lock(m2)
c2--
if(c2==0)
    unlock(m3)
unlock(m1)
```

25
(7)

ניתן לממש עדיפות לכיוון הראשי ע"י שינוי הקוד של הכיוון הנגדי: ושינוי m2 רק שיגרום לעדיפות הכיוון הראשי על פני הנגדי בכך שנמקם אותו לפני m3:

```
drive-2:
    lock(m2)
    lock(m3)
    // drive
    unlock(m3)
    unlock(m2)
```

בצורה הזאת נגרום לכיוון הנגדי לתת זכות ~~[illegible]~~ קדימה לכיוון הראשי ולכן ניתן עדיפות לכיוון הראשי. כי הם ימתינו על m2 וגם על m3.

(כאשר יש נוסע בכיוון הנגדי, אם מגיעים מהכיוון הראשי הם "יכנסו קודם")

שמור דפים שהוחלפו?

הקובץ ששם נשמרים
הדפים להחלפה?
טבלת הדפים?

שאלה 8 (15 נקודות)

א. (5 נקי)

קבוצת מעצבי מערכות הפעלה הציע רעיון הבא כדי להפחית את גודל קובץ החלפה SWAP FILE: לא לשמור את קוד התוכנית בקובץ החלפה בכלל, אלא להביא דפי קוד ישירות מקובץ ההרצה הבינארי בכל פעם שיש צורך בכך. בהנחה שבמערכת לא קיימת הפרדה בין איזור הקוד לאזור הנתונים, איזה מידע נוסף יהיה צורך לשמור בטבלת הדפים?

← כמו demand

(נצטרך לשמור סיווג לכל דף, מאיזה סוג הוא.

תשובה והסבר:

נצטרך בנוסף למידע הרגיל לשמור בטבלת הדפים עבור כל דף את סיווגו - האם הוא דף קוד או דף רגיל, מכיון שבהבאת "דף" נצטרך לדעת לאן ללכת - לקובץ ההרצה הבינארי או לדיסק הקשיח - לאחסון המשני.

ב.(5 נקי)

הסבירו את היתרון הנוסף הקשור למהירות ביצוע של השיטה מעבר לחיסכון במקום.

תשובה והסבר:

מעבר לחיסכון במקום, טעינת דפי הקוד הבינאריים בצורה הזו מעניקה יתרון ומשפרת את מהירות הביצוע מבחינת זמן הביצוע (לא) רק במקום, בעיקר בתוכניות גדולות שמשתמשות בגישה אקראית וקפיצה בין מקומות בקוד, הרי טעינת הקוד כולו לקובץ ההחלפה בתחילת התוכנית גם דורש זמן, וכעת הזמן הזה נחסך.

(בעיקר בתוכנות גדולות עם הרבה קוד)

ג.(5 נקי)

האם אותו הרעיון יכול להתאים לנתונים סטטיים שגם הם נמצאים בקובץ הרצה בינארי?

15
(8)

תשובה והסבר:

לא, מכיוון שנתונים סטטיים נדרשים לתוכנית בכל מקרה וידוע בוודאות שנצטרך להם, לכן עדיף להביאם מראש (כסגנון pre-paging) מכיוון שהצורך בהם מובטח וכך נחסוך שגיאות דף, כאשר בשונה מדפי הקוד שהגישה אליהם דינאמית ולא סטטית.

נא לא לכתוב בשוליים

1KB = 2^{10} 4KB = 2^{12}

שאלה 9 (15 נקודות)

א. (4 נקי)

הוצע רעיון הבא לייעול מערכת קבצים: שטח מחיצה PARTITION יחולק ל-2 אזורים בגודל שווה.

באזור הראשון גודל הבלוק יהיה 4Kbyte ובאזור השני 1Kbyte.

מה היתרון של השיטה החדשה מבחינת ניצול שטח האכסון?

תשובה והסבר:

לשיטה ישנו יתרון משמעותי בכך שהיא באה להפחית פרגמנטציה, הן חיצונית והן פנימית.

כעת ניתן לומר כל הקצאה לבחור בלוקים בגודל המתאים לצרכינו בהתאם לגודל הקובץ/תהליך. להקצאות קטנות נקצה בלוקים קטנים - ונגדל אותם טוב, מה שיפחית פרגמנטציה פנימית הנובעת מבלוקים גדולים.

מכיוון שלא כל הבלוקים גדולים, וחצי מהם קטנים - זה גם יפחית פרגמנטציה חיצונית על הדיסק כולו, חיסרון עיקרי של בלוקים גדולים, שבחלוקה המוצעת יכול להימנע.

ב. (4 נקי)

האם החלק הראשון של קובץ מסוים יכול להיות באזור הראשון, החלק השני באזור השני והחלק השלישי שוב באזור הראשון?

ההסבר אינו נכון

תשובה והסבר:

~~לא~~ הדבר יוצר בעיה מכיוון שמחיצות מהוות הפרדה לוגית של שטח הדיסק כך שמערכות קבצים "מבודדות" יכולות להתנהל במחיצות שונות. קובץ לא יכול להיות מחולק בין 2 מחיצות, וגם משתמשים ב-inode לכל ודאי שלא כך הרי מס' i-node הוא "יחודי" למחיצה.

ג. (7 נקי) $2^6 \cdot 2^{20} = 2^{26}$

בהנחה שגודל המחיצה 64MByte (1Kbyte=2^10Byte, 1MByte=2^20Byte),

תארו את המהלך היעיל ביותר לתרגום מספר הבלוק למיקום בדיסק (מספר הסקטור).

בשביל פשטות החישוב הניחו שגודל הסקטור 1Kbyte.

12
(9)

תשובה והסבר:

כדי לתרגם בלוק למיקום בדיסק, על ה-MMU לחלק (נניח) את הכתובת הוירטואלית. נבחין בין המחיצות ע"י בדיקת הסיבית העליונה - ה MSB של הכתובת, מאחר וגודלי המחיצות שווים - מובטח לנו כי החצי הראשון מתחיל ב-0 והשני ב-1.

בהתאם לסיבית נחלק את הכתובת - 0: 12 סיביות להיסט, 14 סיביות לדף.

1: 10 סיביות להיסט, 16 סיביות למס' דף.

ה-MMU "יגש לטבלת הדפים בהתאם למס' הדף, יחליף במס' המסגרת ויוסיף את ההיסט. ידוע שגודל הכתובת הוא 26 סיביות (+1 לסימון המחיצה, בסה"כ 27) מכיוון שגודל מחיצה נתון - 2^{26}.

נא לא לכתוב בשוליים

חלק ג (15 נקודות)

ענו על **שלוש שאלות** מבין השאלות שלפניכם על פי הסמסטר בו למדתם.

שאלות 10 ו-11 - מיועדות לכלל הסטודנטים.

שאלה 12 - מיועדת לסטודנטים מסמסטר 2025א.

שאלה 13 - מיועדת לסטודנטים מ-2024.

(5 נקודות לשאלה; 15 נקודות לחלק זה).

שאלה 10 - מיועדת לכלל הסטודנטים

האם ביצוע גישה להתקן קלט בעקבות READ יתבצע במצב משתמש USER MODE או במצב גרעין KERNEL MODE ?

תשובה והסבר :

הגישה תתבצע במצב גרעין כמובן. גישה להתקני קלט-פלט (I/O) הינו פעולה מיוחסת וצריכה פיקוח של מערכת ההפעלה. לכן כאשר מתבצע Read, קריאת המערכת תציב את הערכים הרצויים ברגיסטרים המתאימים, ותבצע Trap ע"מ להעביר את השליטה לגרעין המערכת אשר רק לו הרשאה לגישה להתקני ה-I/O וביצוע פעולה זו המוגדרת כפעולה מיוחסת.

5
(10)

שאלה 11 - מיועדת לכלל הסטודנטים

מה יהיה הפלט אחרי הרצת הקוד הבא במערכת XV6 שתומכת בקונטיינרים?
ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת הקוד היה 13 .
ניתן להניח שכל קריאות מערכת מצליחות.

נוצר תהליך חדש, 14.

```
if (fork() == 0) {
   unshare(PID_NS);
   pid = fork();   15 \ 1
   if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n",
pid);
   else printf (stdout, " Child's perspective on his parent: PID=%d \n",
getppid());    0
  }
else  (stdout, "Parent's perspective on his child: PID=%d \n", pid);
```

14

14

(המשך השאלה בעמוד הבא)

תשובה והסבר:

5
(11)

ההדפסות לפי סדר השורות:

Parents... 15

Child... 0

Parents... 14

הסבר: התהליך הראשון שנוצר (14) הוא במרחב הגלובלי, לפני ה-unshare. לכן רואה את בנו כ-15.
הוא יוצר ילד חדש local pid=1, כלומר init של הקונטיינר, ~~[illegible]~~
~~[illegible]~~ ו-getppid עבור init מחזיר 0. ומדפיס 0. הסבא רואה את הבן גלובלי,
ומדפיס 14. ☺

שאלה 12 - מיועדת לסטודנטים מסמסטר 2025א

הסבירו היטב את תפקידם של פקודות unshare(MOUNT_NS) ו mount במערכת XV6 שתומכת בקונטיינרים.

במיוחד התייחסו למשמעות פקודת unshare(MOUNT_NS) ללא ביצוע פקודת mount .

תשובה והסבר:

5
(12)

פקודת unshare משכפלת את מערכת הקבצים של התהליך שקרא לה
ומבודדת את מערכת הקבצים שלו מהמקורי. כעת הוא יוכל להגדיר mount-point
חדשה ע"י ביצוע mount, ובכך להשלים את הבידוד מהמע' הקודמת
בעזרת קבצים שיהיו שייכים רק לו ולא למערכת הקודמת. אם מבצעים unshare
ללא mount, יתקבל שכפול של מע' הקבצים ~~[illegible]~~ הקודמת, וזהו.

שאלה 13 - מיועדת לסטודנטים מ-2024

תהליך נתון שרץ במערכת הפעלה אורחת שרצה במכונה וירטואלית על מערכת מארחת. קיים צורך להריץ את התהליך הזה בקונטיינר. לאיזו מערכת(מארחת או אורחת) צריך להיות מנגנון תמיכה בקונטיינרים?

תשובה והסבר:

בהצלחה!

נא לא לכתוב בשוליים